

GridRAG 简历与面试问答准备文档

本文档用于把 GridRAG 项目讲清楚、讲深入，并把常见面试追问和后端 / 前端 / RAG 八股文问题映射到项目实现中。适合投递简历前复盘，也适合面试前快速背诵。

一句话介绍

GridRAG 是一个基于 FastAPI、React、Chroma、LangChain 和 Qwen 的基层网格治理智能辅助系统，围绕政策知识问答、事件工单填报、居民档案、走访建议、长期记忆和治理看板构建了完整的业务闭环。系统核心是混合检索 RAG：用 Chroma 向量检索和 BM25 关键词检索召回知识片段，通过 RRF 融合和 BGE Reranker 重排，再用 Prompt 约束模型生成带引用来源的答案。

简历写法

项目名称

GridRAG：基于 RAG 的网格员智能管理与服务辅助系统

推荐简历描述

基于 FastAPI + React + Chroma + LangChain + Qwen 实现面向社区网格治理的智能辅助平台，支持知识库文档上传解析、混合检索问答、SSE 流式输出、来源引用、事件 AI 填报、居民走访建议、分层长期记忆和治理数据看板。本人负责后端 RAG 链路、文档入库、Prompt 工程、长期记忆机制、接口设计和前端问答调试台等核心模块。

推荐简历要点

- 设计并实现 RAG 问答链路：文档解析分块后写入 MySQL 和 Chroma，查询时结合 Chroma 向量召回、BM25 关键词召回、RRF 融合和 BGE 重排，提高政策类问答的召回稳定性。
- 实现 SSE 流式问答接口，前端使用 `@microsoft/fetch-event-source` 增量渲染模型输出，并在回答结束后返回引用来源卡片。
- 设计 Prompt 模板体系，分别约束政策问答、事件填报和走访建议，支持依据不足兜底、引用标注、严格 JSON 输出和隐私保护。
- 参考 Claude Code 记忆机制实现分层长期记忆，支持 `organization / project / personal / local / auto / session` 多层级规则、同 key 覆盖和自动经验沉淀。
- 封装 MCP JSON-RPC 网关，对外暴露记忆写入、记忆检索和联网搜索工具，便于外部智能体复用系统能力。
- 使用 Pytest 覆盖分块、引用来源映射、Prompt 渲染契约、分层记忆和安全工具，使用 Ruff 保持代码风格一致。

面试时的 30 秒版本

这个项目是一个社区网格治理场景下的全栈 RAG 系统。它不是单纯的聊天机器人，而是把政策知识库、居民档案、事件工单、走访记录和统计看板结合起来。技术上后端用 FastAPI，前端用 React，知识库用 MySQL 存结构化元数据和分块文本，用 Chroma 存向量。问答时同时走向量召回和 BM25 召回，用 RRF 融合，再用 BGE 重排，最后由 Qwen 生成带引用的答案。项目还实现了 SSE 流式输出、RAG Debug、AI 工单填报、走访建议和类似 Claude Code 的分层长期记忆机制。

面试时的 2 分钟版本

项目目标是解决基层网格员在政策查询、事件填报、重点人群走访中的效率和准确性问题。系统支持上传政策文件、工作手册、历史工单和典型案例，后端解析 PDF、DOCX、TXT、XLSX、CSV 后进行分块，分块文本和元数据写入 MySQL，向量写入 Chroma。

问答链路中，用户问题进入 FastAPI 后会保存会话历史并加载相关长期记忆。检索侧同时使用 Chroma 向量检索和 BM25 关键词检索，解决语义召回和精确关键词召回各自的短板。两路结果通过 RRF 按排名融合，再由 BGE Reranker 重排。相关性不足时不强行回答，而是提示知识库依据不足。生成侧使用 Jinja2 Prompt，把本地知识片段、分层记忆和可选联网搜索结果注入模型，并要求模型用 [1]、[w1] 这类标记标注来源。后端会根据回答中的引用标记过滤来源卡片。

工程上，前端用 React、Ant Design、TanStack Query 和 Zustand，实现问答窗口、会话切换、RAG Debug、知识库上传和治理看板。后端按 API 层、service 层、RAG 层、ingest 层拆分，使用 SQLAlchemy Async、Redis、Celery、Chroma、LangChain 和 Qwen。项目还实现了分层长期记忆，支持组织、项目、个人、本地、全局、自动经验和会话记忆，并用 key 做覆盖去重，避免上下文膨胀。

项目架构怎么讲

总体架构

可以按四层讲：

1. 前端交互层：React + TypeScript + Ant Design，负责问答、知识库、居民、事件、看板等页面。
2. 后端业务层：FastAPI 提供 REST、SSE 和 MCP JSON-RPC 接口。
3. RAG 编排层：负责文档分块、向量检索、BM25、RRF、重排、Prompt、模型调用和来源映射。
4. 数据存储层：MySQL 存业务数据和知识分块，Chroma 存向量索引，Redis 做缓存和 Celery broker。

数据流

文档入库：

1. 前端上传文件到 `/api/v1/knowledge/upload`。
2. 后端校验文件类型，保存原始文件，写入文档记录。
3. Celery 任务解析文件，生成标准文本块。
4. DocumentChunker 按文档类型和文本长度分块。
5. EmbeddingService 生成向量。
6. MySQL 保存分块文本和元数据，Chroma 保存向量索引。

问答：

1. 前端调用 `/api/v1/chat/ask`，后端通过 SSE 返回流式结果。
2. 后端读取分层记忆和会话相关记忆。
3. 向量检索和 BM25 并行召回。
4. RRF 融合候选，BGE Reranker 重排。
5. Prompt 注入知识片段，Qwen 生成答案。
6. 后端解析引用标记，返回被引用的来源卡片。

核心模块说明

文档入库模块

入口: `backend/app/api/v1/knowledge.py`

核心服务:

- `DocumentParser`: 解析 PDF、DOCX、TXT、XLSX、CSV。
- `DocumentChunker`: 把解析后的文本块切成适合检索的 chunk。
- `EmbeddingService`: 调用 BGE embedding 模型生成向量。
- `ChromaStore`: 按 doc_type 管理 Chroma collection。
- `process_document_task`: Celery 任务入口, 串联解析、分块、向量化和入库。

面试表达:

我没有只把文档塞进向量库, 而是 MySQL 和 Chroma 双写。MySQL 保存文档、chunk 文本和元数据, 方便管理、调试和 BM25 检索; Chroma 保存向量, 用于语义召回。这样删除文档、重建索引、Debug 和来源展示都更容易做。

RAG 检索模块

入口: `backend/app/rag/retriever.py`

核心设计:

- Dense retrieval: 使用 embedding 查询 Chroma。
- Sparse retrieval: 从 MySQL 读取 chunk 文本, 用 jieba 分词和 BM25 计算关键词相关性。
- RRF merge: 按两路召回排名做 reciprocal rank fusion。
- Rerank: 用 BGE reranker 对融合候选重新排序。

面试表达:

向量检索适合语义相近但词不完全一样的问题, BM25 适合政策条文、材料名称、专有词这种关键词匹配。两者分数尺度不同, 所以我没有直接相加, 而是用 RRF 按排名融合, 降低分数不可比的问题。

生成与 Prompt 模块

入口: `backend/app/rag/generator.py`、`backend/prompts`

核心设计:

- Jinja2 管理 Prompt 模板。
- LangChain `ChatPromptTemplate` + `ChatOpenAI` + `StrOutputParser` 调用 Qwen OpenAI 兼容接口。
- 问答 Prompt 明确证据优先级: 本地知识库最高, 联网搜索只做补充, 记忆不能作为政策依据。
- JSON 类任务使用 `generate_json` 提取模型返回中的 JSON object, 再由 Pydantic 校验。

面试表达:

Prompt 不是简单写一句“请回答问题”。我把业务规则写进模板, 包括引用格式、依据不足兜底、资料冲突处理、联网搜索降级、JSON 字段约束和隐私规则。这样可以让模型输出更稳定, 也方便测试 Prompt 是否保留关键约束。

SSE 流式问答模块

入口：`backend/app/api/v1/chat.py`、`frontend/src/hooks/useChatStream.ts`

核心设计：

- 后端 `StreamingResponse` 持续返回 SSE event。
- 每个 event 是 JSON，包括 `chunk`、`sources`、`error`、`done`。
- 前端用 `fetchEventSource` 接收增量内容，逐步拼接 assistant 消息。
- 支持停止生成和错误提示。

面试表达：

流式输出的好处是用户不用等模型完整生成后才看到结果，体验更接近真实 AI 助手。后端只需要按照 SSE 格式返回 `data: xxx\n\n`，前端监听 `message` 事件并按 `type` 处理即可。

长期记忆模块

入口：`backend/app/services/memory.py`

核心设计：

- `organization`：组织级规范。
- `project`：项目级规则。
- `personal`：个人偏好。
- `local`：本地环境约定。
- `global`：跨会话生效的全局手动记忆。
- `auto`：自动沉淀的偏好、项目模式和调试经验。
- `session`：当前会话记忆。

面试表达：

这个模块参考了 Claude Code 的记忆机制。人写的稳定规则放在不同 `scope` 中，越具体优先级越高；全局记忆用于保存跨会话都要生效的辖区背景、用户习惯或长期偏好；AI 自动记忆只补充偏好和经验，不覆盖正式规则。同一主题可以设置 `key`，注入 Prompt 前做覆盖和去重，避免每次都把重复内容塞进上下文。

面试官高频追问

1. 这个项目解决了什么问题？

参考回答：

它解决的是基层网格治理中信息分散、政策查询慢、工单录入重复、走访缺少个性化提醒的问题。传统系统只是 CRUD，这个项目把结构化业务数据和非结构化政策文档结合起来，用 RAG 做政策问答，用 AI 辅助生成工单字段和走访建议，同时保留来源引用，减少模型幻觉风险。

2. 为什么要做 RAG，而不是直接微调模型？

参考回答：

这个场景知识更新频繁，比如政策文件、工作手册、历史工单都会变化。如果微调模型，成本高、周期长，而且很难保证最新内容被模型记住。RAG 更适合这种知识密集、更新频繁、需要来源可追溯的场景。知识更新只需要重新入库和建索引，回答还能带引用，方便网格员核验。

3. 为什么要混合检索？

参考回答：

单纯向量检索对语义相似问题很好，但对政策编号、材料名称、专有名词这类精确关键词有时不稳定。BM25 对关键词精确匹配更敏感，但不理解语义。混合检索能结合两者优点。项目里先分别做 Chroma 向量召回和 BM25 召回，再用 RRF 融合，最后用 reranker 精排。

4. RRF 是什么？为什么不用分数加权？

参考回答：

RRF 是 Reciprocal Rank Fusion，核心思想是根据候选在不同召回列表中的排名来融合，而不是直接用原始分数。公式类似 $1 / (k + \text{rank})$ 。因为向量相似度和 BM25 分数尺度不同，直接加权需要做复杂归一化，而且容易受分数分布影响。RRF 更简单稳健，适合多路召回融合。

5. Reranker 的作用是什么？

参考回答：

召回阶段追求“不要漏”，会保留较多候选；reranker 阶段追求“排得准”，用 query 和候选文本一起打分，判断它们是否真正相关。项目里用 BGE Reranker 对 RRF 融合结果重排，然后按阈值判断是否 grounded，低于阈值就不强行回答。

6. 如何降低模型幻觉？

参考回答：

我从三层控制。第一层是检索质量，用混合检索和重排提高依据相关性；第二层是 Prompt 约束，要求严格基于参考资料回答，依据不足就说明没有足够依据；第三层是来源引用，要求关键事实标注 [1]、[2]，后端再根据引用标记返回来源卡片，让用户能核验。

7. 为什么记忆不能作为政策依据？

参考回答：

记忆可能是用户偏好、历史上下文或经验总结，不一定准确，也不一定具有正式效力。政策问答必须以本地知识库里的政策文件或手册为准。所以 Prompt 明确要求记忆只能作为风格、偏好、辖区背景和历史经验，不能作为办理条件、流程、时限等正式依据。

8. SSE 和 WebSocket 怎么选？

参考回答：

这个项目的聊天生成是服务端向客户端单向推送 token，客户端只需要发起一次问题请求，不需要频繁双向通信，所以 SSE 更轻量，浏览器原生支持也更简单。WebSocket 更适合双向实时通信，比如协同编辑、在线游戏、实时状态同步。

9. 为什么后端用异步 SQLAlchemy?

参考回答:

FastAPI 本身适合异步 IO 场景, 项目里有数据库访问、模型 API 调用、联网搜索等 IO 操作。使用 SQLAlchemy Async 可以避免阻塞事件循环, 提高并发请求下的资源利用率。对于 CPU 密集任务, 比如 embedding 或 reranker, 本地部署时要注意用任务队列或独立服务隔离。

10. Redis 在项目里起什么作用?

参考回答:

Redis 主要用于回答缓存和 Celery broker / result backend。问答缓存可以避免相同问题、相同过滤条件、相同记忆上下文下重复调用模型。Celery 这边用于文档入库任务, 方便未来从 eager 模式切换到真正异步队列。

11. 为什么 MySQL 和 Chroma 都要存?

参考回答:

两者职责不同。MySQL 适合保存结构化业务数据、文档记录、chunk 文本和元数据, 便于事务管理、筛选、删除和 Debug。Chroma 适合向量相似度检索。BM25 也需要从 MySQL 读取 chunk 文本。如果只存向量库, 业务管理和精确关键词检索都会变麻烦。

12. 文档删除如何保持一致?

参考回答:

删除文档时需要同时删除 MySQL 文档记录、chunk 记录、原始文件和 Chroma 中对应 document_id 的向量。项目里知识库删除接口会先找到文档, 再调用 ChromaStore 删除向量索引, 删除本地文件, 最后删除数据库记录。

13. 如果检索不到结果怎么办?

参考回答:

项目里不是无论如何都让模型回答。reranker 分数低于阈值并且没有联网搜索结果时, 会返回“未在知识库中找到相关信息”, 建议联系上级部门或查阅原始政策文件。这样比模型硬编答案更适合政务和社区治理场景。

14. RAG Debug 有什么价值?

参考回答:

RAG 出问题时, 不能只看最终答案。Debug 可以看到向量召回、BM25 召回、融合结果、重排分数、Prompt 预览和最终选中的来源。这样能判断问题出在文档解析、分块、召回、重排、Prompt 还是模型生成。

15. 项目有什么不足?

参考回答:

目前还是偏演示和课程设计级别。可以继续完善 Docker Compose 部署、真实权限体系、文档入库异步队列、召回评估集、敏感信息治理、记忆过期和合并策略、模型服务独立部署、RAG 质量指标和生产级日志监控。

RAG 八股文问题

什么是 RAG?

RAG 是 Retrieval-Augmented Generation，检索增强生成。它先从外部知识库检索与用户问题相关的资料，再把资料和问题一起交给大模型生成答案。它适合知识更新频繁、要求可追溯来源、不能只依赖模型参数记忆的场景。

项目关联：

GridRAG 把政策文件、工作手册、历史工单和典型案例解析入库，问答时检索相关 chunk，并要求回答带来源引用。

RAG 和微调有什么区别?

RAG 是把知识放在外部知识库中，通过检索注入上下文；微调是改变模型参数，让模型学习某种风格或领域能力。RAG 更新知识更方便，能提供来源；微调更适合固定风格、固定任务格式或模型能力增强。

项目关联：

基层政策和业务文档经常变动，所以项目选择 RAG，而不是微调。

向量检索的原理是什么?

向量检索先用 embedding 模型把文本映射成高维向量，语义相近的文本在向量空间中距离更近。查询时把问题也转成向量，再计算相似度，找出最相近的文档片段。

项目关联：

GridRAG 使用 BGE embedding 生成向量，Chroma 存储和查询向量。

BM25 的核心思想是什么?

BM25 是经典关键词检索算法，考虑词频、逆文档频率和文档长度归一化。关键词在文档中出现越多、越稀有，分数通常越高，但过长文档会被长度归一化控制。

项目关联：

GridRAG 用 jieba 分词后对 MySQL 中的 chunk 文本构建 BM25，补足向量检索对精确关键词不稳定的问题。

分块为什么重要?

分块会影响召回粒度和上下文质量。块太大，检索结果噪声多，模型上下文浪费；块太小，语义不完整，答案缺上下文。好的分块要保留章节、页码和文档类型等元数据，方便引用和筛选。

项目关联：

GridRAG 的 chunk 带有 `doc_name`、`doc_type`、`page`、`section`、`chunk_index`，用于筛选、引用和 Debug。

为什么需要 reranker?

向量检索和 BM25 召回阶段通常取 top_k，目标是召回足够多的候选。reranker 用更精细的模型重新判断 query 和候选文本的相关性，排序更准，但成本更高，所以放在召回后的小候选集上。

项目关联：

GridRAG 使用 BGE Reranker 对 RRF 融合结果精排。

如何评估 RAG 效果？

常见指标包括 Recall@K、MRR、NDCG、答案正确率、引用准确率、无依据拒答率、人工满意度和端到端延迟。工程上还要看检索耗时、rerank 耗时、模型耗时和缓存命中率。

项目关联：

当前项目已有 retrieval log 和 RAG Debug，后续可以增加标准问题集和离线评估脚本。

后端八股文问题

FastAPI 的优势是什么？

FastAPI 基于 ASGI，天然支持 `async / await`；使用 Pydantic 做数据校验和序列化；自动生成 OpenAPI 文档；类型提示友好，适合构建高性能 API 服务。

项目关联：

GridRAG 用 FastAPI 提供 REST、SSE 和 MCP JSON-RPC 接口。

`async / await` 适合什么场景？

适合 IO 密集型场景，比如数据库访问、HTTP 请求、文件读写、模型 API 调用等。它通过事件循环在等待 IO 时切换任务，提高并发能力。CPU 密集型任务不适合直接放在事件循环里长时间运行。

项目关联：

GridRAG 的数据库访问、联网搜索和模型调用都是 IO 密集型，使用异步接口更合适。

SQLAlchemy ORM 有什么优缺点？

优点是模型和表结构对应清晰，支持关系映射、事务、查询构造和迁移协作。缺点是复杂查询可能不如手写 SQL 直观，错误使用可能导致 N+1 查询或性能问题。

项目关联：

GridRAG 用 SQLAlchemy 2.x Async 管理居民、事件、文档、聊天历史和记忆等数据。

Redis 常见用途是什么？

Redis 常用于缓存、分布式锁、消息队列、排行榜、计数器、会话存储等。它是内存数据库，读写快，但要注意持久化、过期策略和缓存一致性。

项目关联：

GridRAG 用 Redis 做 RAG 回答缓存和 Celery broker。

Celery 适合解决什么问题？

Celery 适合处理耗时任务、异步任务和定时任务，比如文档解析、发邮件、报表生成、模型批处理等。它可以把用户请求和后台处理解耦，避免接口长时间阻塞。

项目关联：

GridRAG 把文档解析、分块、embedding 和向量入库放进 Celery task。

SSE 的格式是什么？

SSE 响应的 `Content-Type` 是 `text/event-stream`，服务端持续返回形如 `data: {...}\n\n` 的文本事件。浏览器接收后触发 `message` 回调。

项目关联：

GridRAG 后端把模型增量输出包装成 `chunk` event，结束时返回 `sources` 和 `done` event。

前端八股文问题

React 中服务端状态和本地状态怎么区分？

服务端状态来自后端接口，比如列表、详情、统计数据，适合用 TanStack Query 管理缓存、刷新和加载状态。本地状态是页面内部交互状态，比如当前输入框、抽屉开关、当前会话 id，适合用 `useState` 或 `Zustand`。

项目关联：

GridRAG 用 TanStack Query 管理接口数据，用 `Zustand` 管理聊天会话和本地消息状态。

为什么用 Zustand？

`Zustand` API 简单，不需要大量样板代码，适合中小型项目管理轻量全局状态。它可以配合 `persist` 中间件把本地会话信息存到 `localStorage`。

项目关联：

GridRAG 用 `Zustand` 存当前聊天会话、历史会话摘要和消息列表。

前端如何处理流式输出？

前端发起请求后监听 SSE message。收到 `chunk` 就把内容追加到当前 assistant 消息；收到 `sources` 就绑定来源卡片；收到 `done` 结束加载状态；收到 `error` 显示错误信息。

项目关联：

GridRAG 的 `useChatStream` 封装了这个过程。

数据库与事务问题

为什么要用 Alembic？

`Alembic` 用于管理数据库 schema 变更。相比启动时自动建表，迁移脚本更适合团队协作和生产环境，能清楚记录每次表结构变化，并支持升级和回滚。

项目关联：

GridRAG 的初始化表和聊天记忆表都有 `Alembic` migration。

如何设计聊天历史表？

聊天历史至少需要 `session_id`、`role`、`content`、`sources`、`created_at`。`session_id` 用于按会话查询，`role` 区分 `user` 和 `assistant`，`sources` 保存回答引用，`created_at` 用于排序。

项目关联：

GridRAG 的 `ChatHistory` 就是按这个思路设计，并对 `session_id + created_at` 建索引。

如何设计长期记忆表？

长期记忆需要 session_id、content、memory_type、metadata、usage_count、last_used_at、created_at、updated_at。metadata 可以扩展 scope、key、category 等字段，避免频繁改表。

项目关联：

GridRAG 的分层记忆机制复用了 `metadata_json` 存 scope、key 和 category；为了支持 `__gridrag_memory_scope__:organization`、`__gridrag_memory_scope__:global` 这类保留 scope id，新增迁移把 `chat_memories.session_id` 放宽到 64 位。

Prompt 工程问题

如何让模型稳定输出 JSON？

需要在 Prompt 中明确“只输出合法 JSON，不要 Markdown、代码块、注释或额外文字”，同时给字段名、字段类型、枚举值、长度限制和示例。后端仍要做 JSON 提取和 Pydantic 校验，不能只相信模型。

项目关联：

GridRAG 的事件填报和走访建议都使用 JSON Prompt，再用 Pydantic schema 校验。

如何处理联网搜索和本地知识库冲突？

政务和业务系统里，本地知识库通常是正式依据，联网搜索只能补充。Prompt 应明确优先级：本地资料高于联网资料；如果冲突，要提示用户核对原始文件，不要强行合并。

项目关联：

GridRAG 的问答 Prompt 明确本地知识库优先，联网搜索只作为补充。

为什么要测试 Prompt？

Prompt 是系统行为的一部分，改动 Prompt 可能导致引用规则、隐私规则、JSON 约束丢失。用测试检查模板渲染后是否包含关键约束，可以避免后续维护时无意破坏行为。

项目关联：

GridRAG 新增了 `test_prompts.py`，测试问答、事件填报和走访建议模板的关键规则。

项目难点与回答模板

难点 1：检索结果不稳定

回答模板：

最开始只用向量检索时，对语义相似问题还可以，但遇到政策材料名、条款关键词、专有名词时不稳定。所以我增加了 BM25 关键词召回，用 RRF 融合两路结果，再用 BGE Reranker 精排。这样既能处理语义匹配，也能保留关键词精确匹配能力。

难点 2：模型容易无依据回答

回答模板：

我从检索阈值、Prompt 约束和引用后处理三方面控制。重排分数低时直接返回依据不足；Prompt 明确不能编造政策条款；答案中必须用引用标记，后端只返回实际引用过的来源。这样能让用户知道答案来自哪里。

难点 3：记忆容易膨胀或污染回答

回答模板：

我把记忆拆成规则层、全局层和经验层。组织、项目、个人、本地规则是人写的，全局记忆跨会话生效，自动记忆只沉淀偏好和经验。同主题用 key 覆盖去重，注入 Prompt 前压缩成带标签片段。同时 Prompt 明确记忆不能作为政策依据，避免记忆污染正式问答。

难点 4：文档入库和业务数据一致性

回答模板：

文档相关数据分布在原始文件、MySQL 和 Chroma。入库时先写文档记录，再后台任务解析分块、写 chunk、写向量；删除时要删除 Chroma 向量、本地文件和数据库记录。这样能保证知识库页面、检索链路和向量索引的一致性。

面试官可能继续深挖的问题

如果文档很多，BM25 每次从 MySQL 读全量 chunk 会不会慢？

参考回答：

会，这是当前实现偏演示化的地方。数据量上来后可以把 BM25 索引常驻内存或换成 Elasticsearch / OpenSearch；也可以按 doc_type、更新时间做增量索引，避免每次请求重新构建 BM25。

如果 embedding 模型很慢怎么办？

参考回答：

可以批量 embedding、GPU 部署、独立 embedding 服务、任务队列限流和缓存。文档入库是离线任务，可以异步慢慢处理；查询 embedding 是在线链路，需要重点优化延迟。

如果模型输出没有引用标记怎么办？

参考回答：

项目里做了兜底：如果回答里没有引用标记，后端会保留已选上下文作为来源。但更好的做法是继续强化 Prompt，或在生成后做引用补全 / 答案校验。

如何做权限？

参考回答：

当前演示环境默认 `AUTH_DISABLED=true`。生产环境应接入用户、角色、部门、数据范围和审计日志。比如网格员只能看自己辖区居民和事件，管理员能看全局统计，知识库上传和删除需要更高权限。

如何部署？

参考回答：

可以用 Docker Compose 编排 MySQL、Redis、后端、前端和 Chroma 持久化目录。模型可以调用云端 Qwen，也可以把 embedding / reranker 独立成服务。生产环境还要配置 Nginx、HTTPS、日志采集和备份。

可以反问面试官的问题

- 如果这个系统上线到真实政务场景，您更关注回答准确率、响应延迟还是权限审计？
- 对 RAG 系统来说，贵团队更常用向量库方案，还是 Elasticsearch / OpenSearch 混合检索方案？
- 如果要把这个项目扩展成生产系统，您认为第一优先级应该是评估体系、权限体系还是部署运维？

快速背诵版

项目是什么：

GridRAG 是一个社区网格治理场景的全栈 RAG 系统，支持政策知识问答、AI 工单填报、居民走访建议、长期记忆和治理看板。

核心技术：

FastAPI、React、MySQL、Redis、Celery、Chroma、LangChain、Qwen、BGE embedding、BGE reranker、BM25、RRF、SSE。

核心亮点：

混合检索、RRF 融合、BGE 重排、来源引用、RAG Debug、严格 Prompt、分层长期记忆。

最能体现能力的点：

不是简单调用大模型，而是做了完整 RAG 工程链路：文档入库、检索、重排、Prompt、流式输出、引用后处理、记忆管理、调试和测试。

项目不足：

当前更适合本地演示和二次开发，生产化还需要权限体系、Docker Compose、召回评估集、BM25 索引优化、敏感信息治理和观测指标。